Honigwabenlabyrinth

Algorithmische Geometrie 2025

Lucas Stramke, Toni Nguyen - 04.07.2025

HoneyBadger-HMG



Umsetzungsgedanken

I. Datendarstellung
II. Labyrinth generieren
III. Labyrinth interessant machen
IV. Lösungsgraph erzeugen
V. Erzeugen der C++ Datei
VI. Verwendung als CLI-Tool

I. Datendarstellung

Vom letzen Mal

- **Wabe:**
  - ID
  - Definierende Kanten - 6 [leftAbove, rightAbove, leftSide, rightSide, leftBottum, rightBottum]
  - Nachbarzellen - maximal 6
- **Kante:**
  - Punkt1, Punkt2
  - isWall
  - isDestructible
- **Punkt:**
  - X,Y Koordinate

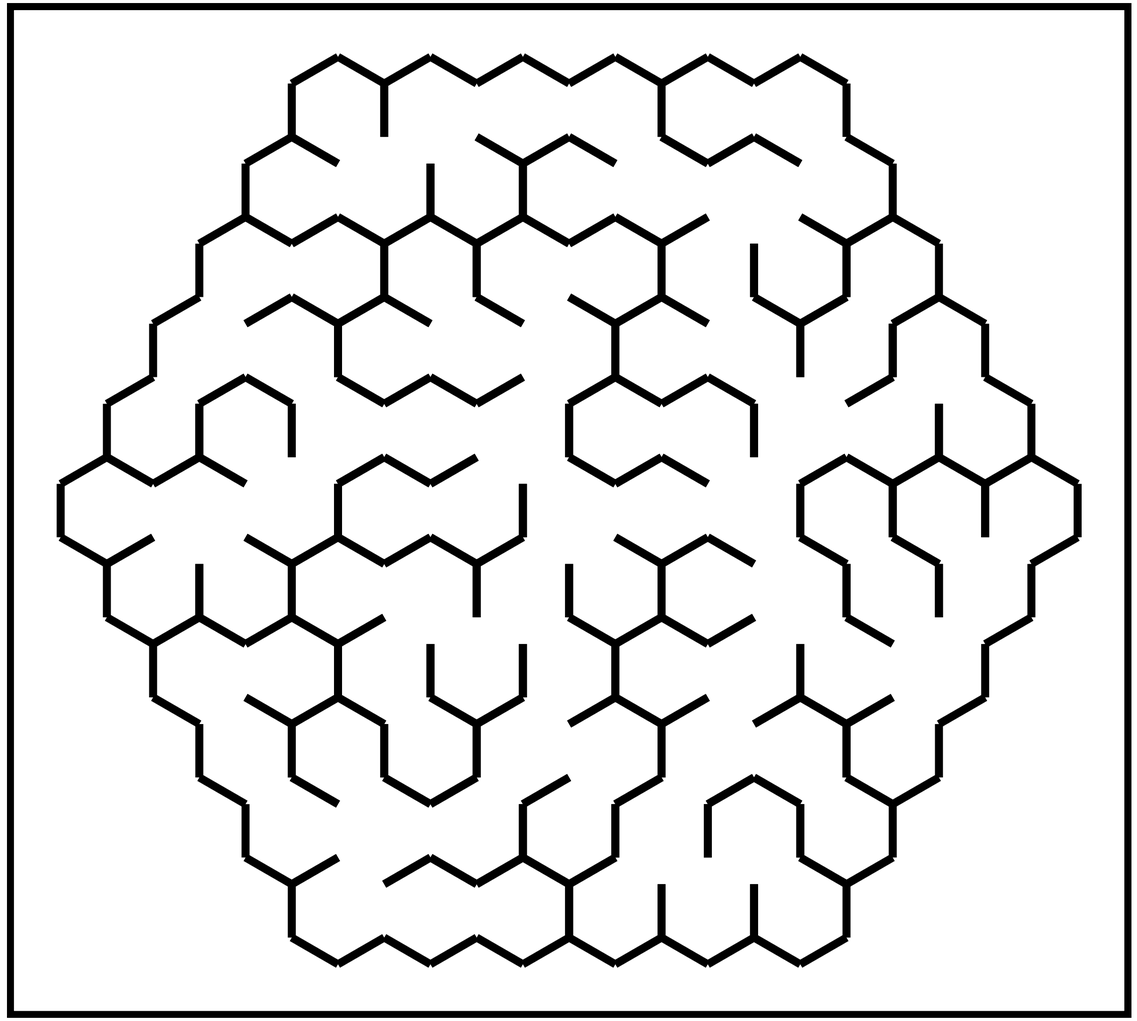
II. Labyrint generieren

Labyrinth als Spannbaum

Spannbaum auf Honigwaben-Gitter

Ziel:
Spannbaum auf Honigwaben-Gitter erzeugen

- **Algorithmus:** Randomisierter Prim-Algorithmus
- **Struktur:** Honigwaben-Gitter
- Vorgehen:
  - Startknoten zufällig wählen
  - Nachbarn zufällig erweitern (Kantenpriorität)
  - Zyklusvermeidung durch Baumstruktur
- Ergebnis:
  - Vollständiger Spannbaum
  - Visuelle Ausgabe als SVG



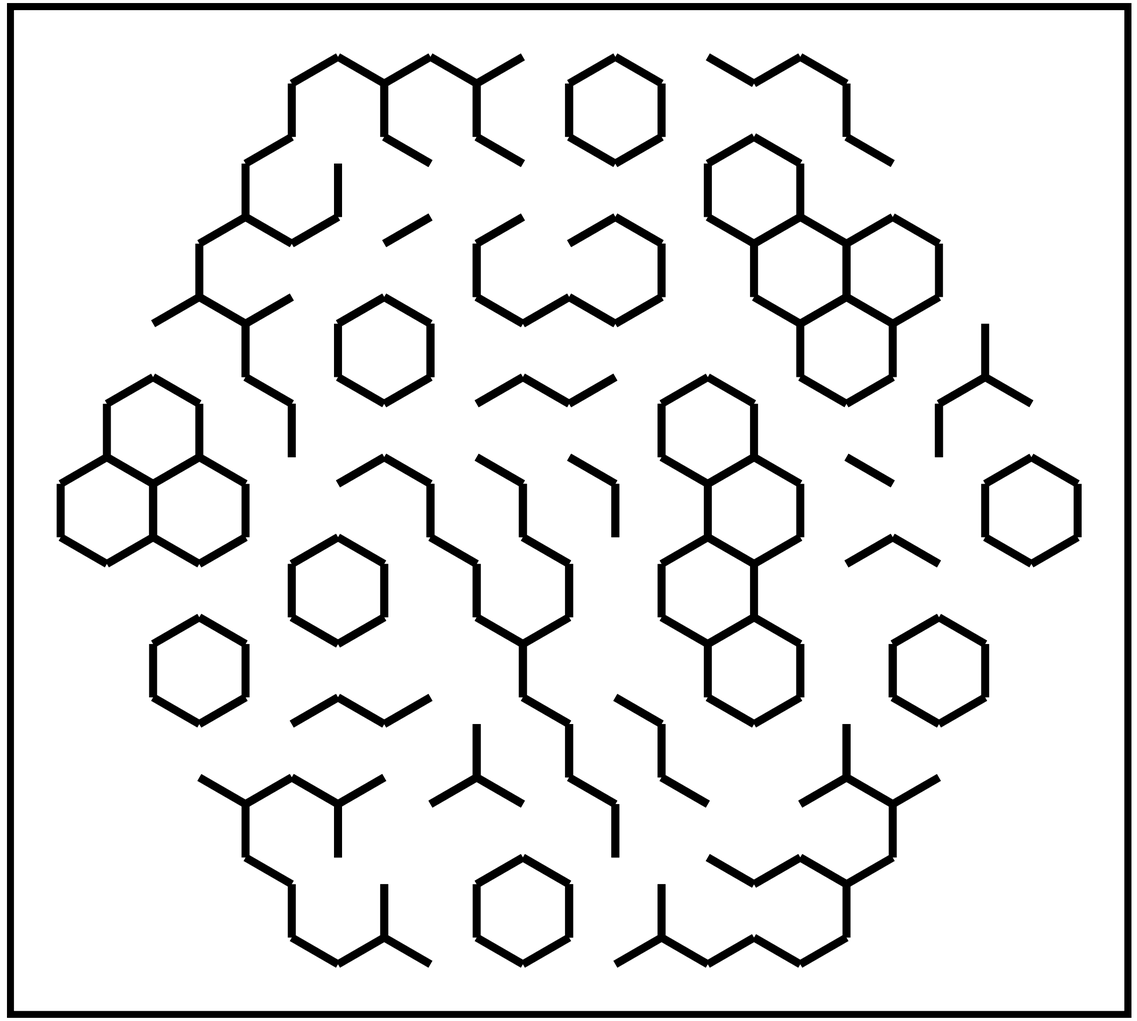
III. Labyrinth interessant machen

Der „HoneyBadger“

„HoneyBadger“-Prozess (Maze-Manipulation)

Ziel:
Veränderung zufälliger Zellen im Labyrinth

- **Parameter:** Prozentwert für zu verändernde Zellen
- Vorgehen:
  - Zelle zufällig auswählen
  - Alle Kanten der Zelle:
    - Unzerstörbar machen
    - Zu Wänden machen
  - Für benachbarte Zellen:
    - Entferne benachbarte zerstörbare Wände
- Effekt:
  - Schafft unregelmäßige Blockaden
  - Erhöht Labyrinth-Komplexität und Variation



IV. Lösungsgraph erzeugen

Lösungswege

Lösungsgraph-Erzeugung aus Honigwaben-Labyrinth

Ziel:
Struktur zur Darstellung aller begehbaren Pfade im Labyrinth

Datenstruktur:

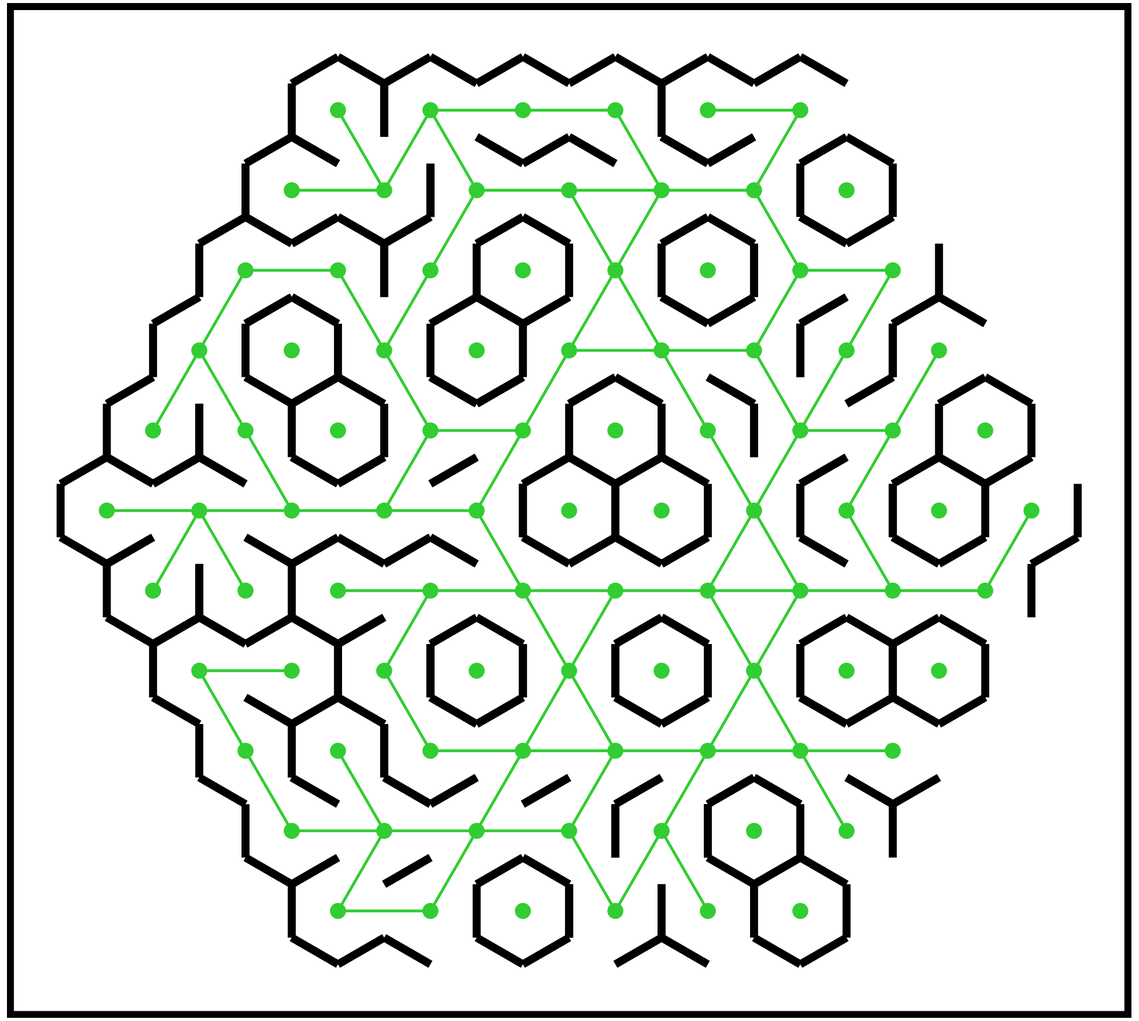
- **Knoten:** Zentrum jeder begehbaren Zelle
- **Kanten:** Verbindungen zu direkt erreichbaren Nachbarzellen

Vorgehen:

1. Alle Zellen als Graph-Knoten anlegen
2. Nur gegenseitig erreichbare Nachbarn verbinden
3. List<SolutionGraphNode>

Nutzen:

- Grundlage für Pfadsuche, Analyse & Visualisierung
- Leicht exportier- & darstellbar



V. Erzeugen der C++ Datei

Ziel:
Automatisierte Erzeugung einer C++ Datei aus SVG und Lösungsgraph

Umsetzung:

- Kombination von SVG-Daten und Lösungsgraph-Struktur in einem Parser zu C++ Code

Ergebnis:

- Die Datei maze_generator.cpp

```cpp
#include <iostream>
#include "maze_generator.h"

void MazeGenerator::generateMaze()
{
    {
        Mazepolygon poly;
        poly.coordinates.push_back({-17.32f, 10.00f});
        poly.coordinates.push_back({0.00f, 20.00f});
        polygons.push_back(poly);
    }
    {
        Mazepolygon poly;
        poly.coordinates.push_back({0.00f, 20.00f});
        poly.coordinates.push_back({17.32f, 10.00f});
        polygons.push_back(poly);
    }
    {
        Mazepolygon poly;
        poly.coordinates.push_back({17.32f, 10.00f});
        poly.coordinates.push_back({17.32f, -10.00f});
        polygons.push_back(poly);
    }
    {
        Mazepolygon poly;
        poly.coordinates.push_back({17.32f, -10.00f});
```

VI. Verwendung als CLI-Tool

CLI Parameters

Parameter	Description	Example
`-t`	Maze type (e.g., `honeycomb`)	`-t honeycomb`
`-r`	Rank (size of the maze, >0)	`-r 3`
`-el`	Edge length (>0)	`-el 2`
`-cp`	Percentage of cells to modify (0.0–100.0)	`-p 50.0`
`-p`	Path for generated C++ file	`-p ./path/path`

Beispiel:

```
java -jar target/honeyBadger-hmg-1.0-SNAPSHOT.jar -t honeycomb -r 3 -el 2 -cp 20.0 -p ./c++
```

Terminal Ausgabe:

- **SVG file created:** /Users//HoneyBadger-HMG/honeybadger-hmg/svg/normal.svg
- **SVG file created:** /Users//HoneyBadger-HMG/honeybadger-hmg/svg/badged.svg
- **SVG file created:** /Users//HoneyBadger-HMG/honeybadger-hmg/svg/sg.svg
- **SVG file created:** /Users//HoneyBadger-HMG/honeybadger-hmg/svg/merged.svg
- **C++ file "maze_generator.cpp" created:** /Users//HoneyBadger-HMG/honeybadger-hmg/./c++/maze_generator.cpp

VIELEN DANK FÜR EURE AUFMERKSAMKEIT!!